FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS
Ingeniería en Ciberseguridad

PROGRAMACIÓN III (ISWZ2102)

INFORME TÉCNICO

AVANCE DEL PROYECTO, PROGRESO 1

Análisis, diseño y selección de la solución

TEMA: SECR, Sistema de Evaluación de Conducción Responsable

Priorización del riesgo de conductores mediante estructuras de datos y algoritmos en Java

ESTUDIANTES (GRUPO 4):

Mateo Vasquez
Alex Palma
Martín Jurado
Amelio Macas
Israel Zurita

Período académico 2027-1
QUITO, ECUADOR
Octubre de 2026

Contenido

Resumen ejecutivo....2
1. Formulación del problema....3
1.1 Contexto y necesidad....3
1.2 Actores y grupos involucrados....3
1.3 Causas y consecuencias....4
1.4 Datos y procesos principales....4
1.5 Variables relevantes y restricciones....4
1.6 Relación con factores globales, sociales, culturales y económicos....5
1.7 Enunciado del problema....5
2. Alcance y módulos funcionales....6
2.1 Alcance....6
2.2 Módulo M1: Gestión de conductores y licencias....6
2.3 Módulo M2: Registro y análisis de eventos e infracciones....6
2.4 Módulo M3: Puntaje y priorización de riesgo....7
2.5 Módulo M4: Recuperación de puntos (cursos, inscripciones y pruebas)....7
2.6 Módulo M5: Alertas, reportes y operativos de control....8
2.7 Matriz de responsabilidades....9
3. Modelo orientado a objetos y diagrama UML preliminar....10
3.1 Herramienta y criterios....10
3.2 Diccionario de las clases de dominio principales....10
3.3 Justificación de herencia, composición y asociaciones....12
3.4 Correspondencia entre módulos, clases y estructuras....13
4. Alternativas basadas en estructuras de datos y algoritmos....16
4.1 Detalle de complejidad por operación....17
5. Selección y fundamentación de la mejor alternativa....18
5.1 Evidencia experimental....18
5.2 Análisis por criterios....19
5.3 Decisión....19
6. Plan preliminar de implementación y gestión....20
6.1 Módulos y componentes asignados....20
6.2 Estructura del repositorio Git....20
6.3 Cronograma hacia el Progreso 2....21
6.4 Riesgos técnicos y acciones de mitigación....21
6.5 Declaración de uso de herramientas de inteligencia artificial....22
Referencias....23
Anexo A. Reproducción de los resultados....23

Resumen ejecutivo

El presente informe documenta el Progreso 1 del proyecto de Programación III. El equipo retoma **SECR (Sistema de Evaluación de Conducción Responsable)**, desarrollado el semestre anterior como una aplicación Java con interfaz Swing orientada principalmente al registro (CRUD) de conductores, infracciones, cursos y licencias, y lo replantea como un problema de **estructuras de datos y algoritmos**: mantener actualizado el ranking de conductores de mayor riesgo, consultar el historial de infracciones por rangos de fechas, asignar cupos en cursos de recuperación y planificar operativos de control sobre el mapa vial.

Por requisito del curso, la solución usa exclusivamente **pilas, colas, árboles y grafos**, implementados por el equipo: árboles AVL para los índices, una pila para deshacer registros, una cola FIFO para los cupos, una cola de prioridad para la caducidad de licencias y un grafo ponderado con BFS, DFS y Dijkstra. Las tablas hash heredadas del semestre anterior fueron reemplazadas por árboles AVL.

Para el componente crítico (priorización por riesgo) se comparan dos alternativas competitivas: **A**, dos árboles AVL, y **B**, una cola de prioridad con actualización perezosa. Ambas se prototiparon, se verificó con 29 pruebas automáticas que dan los mismos resultados y se midieron con 100 000 conductores. B actualiza puntajes unas 2,8 veces más rápido; A responde el top-10 unas 9 veces más rápido, entrega resultados ordenados y usa menos memoria. Se selecciona la **Alternativa A**. El informe cierra con el plan de implementación, la estructura del repositorio, los riesgos técnicos y la declaración de uso de inteligencia artificial.

1. Formulación del problema

1.1 Contexto y necesidad

Los siniestros de tránsito son un problema de salud pública a escala mundial. La Organización Mundial de la Salud estima que provocan alrededor de **1,19 millones de muertes al año**, que son la principal causa de muerte de niños y jóvenes de 5 a 29 años y que nueve de cada diez de esas muertes ocurren en países de ingresos bajos y medianos (World Health Organization [WHO], 2023). Ecuador refleja esta realidad: según datos de la Agencia Nacional de Tránsito (ANT), en 2024 se registraron **21 220 siniestros y 2 302 personas fallecidas**, lo que equivale aproximadamente a una muerte cada 3,8 horas; las causas más frecuentes fueron la conducción desatenta, el exceso de velocidad y el irrespeto a las señales de tránsito (El Universo, 2025).

Para desincentivar las conductas de riesgo, el país aplica un **sistema de licencias por puntos**: el conductor parte de un saldo de 30 puntos, cada contravención descuenta puntos y, al agotarlos, la licencia se suspende; los puntos pueden recuperarse mediante cursos autorizados (Asamblea Nacional del Ecuador, 2008). La evidencia internacional indica que estos sistemas reducen los siniestros, fallecidos y lesionados entre un 15 % y un 20 % al inicio, pero que el efecto se diluye en menos de dieciocho meses si no se acompaña de control y seguimiento efectivos (Castillo-Manzano & Castro-Nuño, 2012); el análisis econométrico del caso español confirma la necesidad de sostener la vigilancia (Castillo-Manzano et al., 2010).

Ese seguimiento exige información actualizada que se transforme en acciones oportunas. El semestre anterior el equipo construyó SECR como un sistema de registro; al analizarlo con criterios de Programación III se concluyó que las operaciones de mayor valor son las de **procesamiento**: saber en todo momento quiénes son los conductores de mayor riesgo, consultar el historial por periodos, priorizar las licencias por caducar y asignar cupos en los cursos. La versión anterior las resolvía recorriendo o reordenando colecciones completas, algo aceptable con decenas de registros, pero no con decenas de miles.

1.2 Actores y grupos involucrados

Tabla 1. *Actores del problema y su interés en la solución*

Actor	Rol en el problema	Interés principal
Agente u operador de tránsito	Registra infracciones y consulta el historial del conductor.	Registro rápido y validado; consulta inmediata por licencia.
Analista o administrador de la autoridad de tránsito	Configura catálogos, revisa rankings y genera reportes.	Identificar conductores de alto riesgo y licencias por caducar para intervenir a tiempo.
Conductor	Sujeto evaluado; acumula infracciones, rinde pruebas y renueva su licencia.	Conocer su puntaje, recuperar puntos y renovar sin demoras.
Centro de capacitación o instructor	Dicta los cursos de recuperación y aplica las pruebas.	Asignación ordenada de cupos y evaluación objetiva.
Sociedad (peatones, ciclistas, otros conductores)	Beneficiario indirecto de vías más seguras.	Reducción de siniestros por conductas reincidentes.

1.3 Causas y consecuencias

Causas. (1) Información de infracciones, cursos y renovaciones dispersa; (2) puntaje recalculado manualmente o por lotes; (3) ausencia de un mecanismo que muestre de inmediato a los conductores de mayor riesgo; (4) consultas resueltas con recorridos lineales y reordenamientos completos; y (5) control reactivo de la caducidad de licencias.

Consecuencias. La intervención sobre conductores reincidentes llega tarde, lo que debilita el efecto disuasorio que el sistema de puntos busca (Castillo-Manzano & Castro-Nuño, 2012); circulan conductores con licencias vencidas; los cupos de los cursos se asignan sin un criterio verificable; y los reportes se vuelven lentos a medida que crece la base de datos.

1.4 Datos y procesos principales

Datos: conductor (cédula, nombre, fecha de nacimiento, estado, puntaje); licencia (número, categoría B, C o E, vigencia de 5 años); tipo de evento (puntos base); evento (fecha, ubicación, gravedad de 1 a 5); curso (puntos recuperables, porcentaje mínimo, cupo); pregunta, prueba, renovación y alerta.

Proceso central: registro del conductor → registro de infracción → cálculo de la deducción → recálculo del puntaje y reclasificación del riesgo → generación de alertas → inscripción en un curso → prueba de recuperación → acreditación de puntos → renovación de la licencia cuando corresponde.

1.5 Variables relevantes y restricciones

Tabla 2. *Variables del problema*

Variable	Dominio / fórmula	Uso
Puntaje de conducción	Real en [0, 100]. Puntaje = 100 − Σ deducciones + recuperación + bonificación	Clasificación de riesgo y prioridad
Deducción por evento	puntosBase(tipo) × gravedad, gravedad ∈ {1,…,5}	Actualización del puntaje
Nivel de riesgo	Verde ≥ 80; amarilla ≥ 60; naranja ≥ 40; roja < 40	Alertas y semáforo
Días para vencer	fechaVencimiento − hoy (aviso con 30 días)	Priorización de caducidad
Bonificación	5 puntos por año sin infracciones	Incentivo al buen comportamiento
Aprobación de prueba	Nota ≥ 70 % (configurable por curso)	Recuperación de 20 o 30 puntos
Volumen	n conductores (10^3 a 10^5), m eventos, k tamaño del ranking	Análisis de complejidad y escalabilidad

Escala del puntaje. SECR usa un índice normalizado de 0 a 100 en lugar de los 30 puntos legales para ganar granularidad en la clasificación del riesgo; la conversión es lineal (100 puntos SECR equivalen a 30 puntos legales).

Restricciones del dominio: el conductor debe tener al menos 18 años y una cédula válida (algoritmo de dígito verificador módulo 10); el número de licencia es único con formato B-001 a E-999999; la licencia solo se renueva si está vencida, el conductor está activo y tiene al menos 50 puntos; un evento no puede tener fecha futura ni anterior a cinco años; al llegar a 0 puntos el conductor queda inactivo; no se elimina un conductor con historial.

Restricciones del proyecto: Java 17 o superior sin librerías externas; estructuras de datos limitadas a pilas, colas, árboles y grafos (sin tablas hash); reutilización del código anterior; datos en memoria en esta fase; equipo de cinco integrantes; escalar a 10^5 conductores en un computador personal.

1.6 Relación con factores globales, sociales, culturales y económicos

Global y social: la reducción de muertes por siniestros de tránsito es una meta de la Agenda 2030 (ODS 3.6) y del Decenio de Acción para la Seguridad Vial 2021-2030, que plantea reducirlas a la mitad para 2030 (WHO, 2023). **Cultural:** varias de las causas principales, como la conducción desatenta o el uso del celular, son conductas normalizadas; SECR combina la sanción con incentivos (bonificación por buen comportamiento y cursos de recuperación) para reforzar una cultura de conducción responsable. **Económico:** los siniestros generan costos de atención médica y pérdida de productividad para las familias y el Estado (WHO, 2023); priorizar la intervención sobre los conductores de mayor riesgo optimiza el uso de recursos de control. No se identifica una relación ambiental directa y justificable, por lo que no se considera.

1.7 Enunciado del problema

¿Cómo organizar en memoria la información de conductores, infracciones y cursos para que las operaciones críticas del sistema de puntos (búsqueda, priorización por riesgo, consulta por periodos y asignación de cupos) respondan en tiempo constante o logarítmico a medida que crece el número de registros, manteniendo la coherencia con las reglas de negocio?

2. Alcance y módulos funcionales

2.1 Alcance

Incluye: gestión de conductores y licencias; registro y análisis de infracciones; cálculo y priorización del riesgo; cursos de recuperación con cola de inscripción y pruebas; alertas, reportes y planificación de operativos de control sobre un mapa vial. Se mantienen la interfaz gráfica Swing y la consola del semestre anterior. Por requisito del curso, **todas las estructuras de datos de la solución son pilas, colas, árboles o grafos**, implementados por el equipo; no se usan tablas hash (HashMap, HashSet, LinkedHashMap) ni árboles de prefijos (Trie).

No incluye en esta fase: persistencia en base de datos (se evaluará archivo CSV en el Progreso 2), integración con sistemas reales de la ANT, georreferenciación con mapas reales, autenticación de usuarios ni despliegue web.

En cada módulo se marcan con **[ALG]** las operaciones algorítmicas propias del dominio (búsqueda, recorrido, priorización, filtrado, asignación o cálculo de rutas), que van más allá del CRUD.

2.2 Módulo M1: Gestión de conductores y licencias

Propósito y actor	Mantener el padrón de conductores y sus licencias y localizarlos de forma inmediata. Actor principal: operador de tránsito.
Datos	Conductor, Licencia, RenovacionLicencia. Índices: árbol AVL por número de licencia y árbol AVL por nombre normalizado.
Operaciones	• [ALG] Registrar conductor con validación de cédula (dígito verificador módulo 10), edad y formato de licencia, e insertarlo en ambos AVL en O(log n). • [ALG] Buscar por número de licencia en O(log n), con altura del árbol garantizada. • [ALG] Búsqueda por prefijo de nombre o apellido: consulta de rango [prefijo, prefijo + '\uffff'] sobre el AVL de nombres, O(log n + k). • [ALG] Filtrar conductores por categoría, estado o licencia vencida mediante recorrido en orden (in-order) del árbol. • Renovar licencia (nueva vigencia de 5 años) y registrar el historial de renovaciones. • Actualizar datos, cambiar estado y eliminar (solo sin historial).
Reglas de negocio	Mayor de 18 años; cédula válida; licencia y cédula únicas; la renovación exige licencia vencida, conductor activo y puntaje ≥ 50; no se elimina un conductor con infracciones, pruebas o renovaciones.
Relación con otros módulos	Provee conductores a M2, M3, M4 y M5. Recibe de M3 el cambio de estado cuando el puntaje llega a 0.
Responsable	**Mateo Vasquez**

2.3 Módulo M2: Registro y análisis de eventos e infracciones

Propósito y actor	Registrar las infracciones, calcular su impacto en puntos y analizar el historial. Actor principal: agente de tránsito.
Datos	EventoConduccion, Infraccion, TipoEvento, Zona. Estructuras: árbol AVL por fecha (IndiceTemporalEventos) y pila de operaciones para deshacer.

Operaciones	• [ALG] Registrar infracción, calcular la deducción (puntos base × gravedad) y apilar la operación. • [ALG] Deshacer el último registro erróneo con una pila (LIFO): revierte la deducción y el índice en O(log n). • [ALG] Consultar eventos por rango de fechas, global o por conductor, en O(log d + k). • [ALG] Ordenar eventos por gravedad y fecha con ordenamiento por mezcla estable, O(m log m). • [ALG] Infracción más frecuente: conteo con un árbol AVL (tipo → contador), O(m log t). • Administrar el catálogo de tipos de evento (crear, editar, activar o inactivar).
Reglas de negocio	Solo conductores activos y con puntos; fecha no futura ni anterior a 5 años; gravedad de 1 a 5; solo tipos activos; un tipo ya usado no se elimina, se inactiva; solo se deshace el último registro de la sesión.
Relación con otros módulos	Usa M1 para obtener el conductor; notifica cada deducción (o su reversión) a M3; alimenta los conteos por zona de M5.
Responsable	**Alex Palma**

2.4 Módulo M3: Puntaje y priorización de riesgo

Propósito y actor	Mantener actualizado el puntaje y el orden de riesgo de todos los conductores. Actor principal: analista de la autoridad de tránsito.
Datos	HistorialConduccion, puntaje actual, NivelAlerta. Estructura seleccionada (sección 5): árbol AVL ordenado por la clave (puntaje, licencia).
Operaciones	• Recalcular el puntaje: 100 − deducciones + recuperación + bonificación, acotado a [0, 100]. • [ALG] Reubicar al conductor en el orden de riesgo tras cada evento o curso: eliminar la clave anterior e insertar la nueva, O(log n). • [ALG] Obtener el top-k de conductores de mayor riesgo con un recorrido en orden desde el mínimo, O(log n + k). • [ALG] Filtrar por semáforo (verde ≥ 80, amarilla ≥ 60, naranja ≥ 40, roja < 40) con una consulta de rango, O(log n + k). • Aplicar la bonificación por buen comportamiento (5 puntos por año sin infracciones).
Reglas de negocio	Puntaje en [0, 100]; con 0 puntos el conductor pasa a inactivo; mayor riesgo = menor puntaje; en empate se ordena por número de licencia para que el resultado sea determinista.
Relación con otros módulos	Recibe actualizaciones de M2 (deducciones) y M4 (recuperaciones); consulta M1; entrega el ranking y los semáforos a M5.
Responsable	**Martín Jurado**

2.5 Módulo M4: Recuperación de puntos (cursos, inscripciones y pruebas)

Propósito y actor	Gestionar los cursos de recuperación, asignar cupos y evaluar a los conductores. Actores: conductor e instructor.

Datos	Curso, Pregunta, Prueba, EvaluacionPrueba e InscripcionCurso (nueva). Estructuras: árbol AVL de cursos por código y una cola FIFO de inscripciones por curso.
Operaciones	• [ALG] Inscribir al conductor al final de la cola FIFO del curso, O(1). • [ALG] Asignar cupos por orden de llegada: desencolar hasta completar el cupo máximo, O(c). • [ALG] Generar una prueba aleatoria con el algoritmo de Fisher-Yates (Durstenfeld, 1964), O(p). • Calificar la prueba y acreditar los puntos si la nota alcanza el mínimo. • [ALG] Reporte del curso con mayor porcentaje de aprobación (conteo en árbol AVL por código). • Administrar cursos y banco de preguntas.
Reglas de negocio	Solo conductores con puntaje menor a 100; curso activo; aprobación ≥ 70 %; recupera 20 (básico) o 30 (avanzado) puntos sin superar 100; un conductor no puede estar dos veces en la misma cola; no se elimina un curso con preguntas.
Relación con otros módulos	Consulta M1; notifica las recuperaciones a M3; entrega estadísticas a M5.
Responsable	**Amelio Macas**

2.6 Módulo M5: Alertas, reportes y operativos de control

Propósito y actor	Convertir los datos en decisiones: alertas, reportes y planificación de operativos de control en las zonas con más infracciones. Actor principal: administrador o analista.
Datos	Alerta, Licencia, Zona, TramoVial y resultados de M1 a M4. Estructuras: cola de prioridad (montículo binario) por fecha de vencimiento y grafo ponderado del mapa vial.
Operaciones	• [ALG] Alertas de caducidad priorizadas por días para vencer con una cola de prioridad (montículo mínimo), O(log n) por alerta. • [ALG] Zonas críticas: top-k de zonas con más infracciones (conteo en AVL + montículo), O(z log z). • [ALG] Ruta del operativo de control desde la base de agentes hasta una zona crítica con Dijkstra, O((V + E) log V). • [ALG] Zonas de influencia: zonas a k tramos de un punto crítico con búsqueda en anchura (BFS), O(V + E). • [ALG] Detectar zonas aisladas del mapa con DFS iterativo (pila), O(V + E). • Reporte individual y reporte estadístico general.
Reglas de negocio	Aviso de caducidad con 30 días de anticipación; alerta roja con puntaje menor a 40; las distancias de los tramos son positivas; una zona es crítica si supera el umbral de infracciones del periodo.
Relación con otros módulos	Consume información de M1, M2, M3 y M4.
Responsable	**Israel Zurita**

2.7 Matriz de responsabilidades

Tabla 3. *Asignación preliminar de responsabilidades (R = responsable, A = apoyo)*

Integrante	M1	M2	M3	M4	M5	Responsabilidad transversal
Mateo Vasquez	R		A			Árbol AVL genérico y sus pruebas
Alex Palma		R			A	Pila de deshacer y consultas por rango
Martín Jurado	A		R			Integración del ranking con la interfaz
Amelio Macas				R	A	Cola FIFO y administración del repositorio
Israel Zurita		A		A	R	Grafo vial, benchmark y pruebas

3. Modelo orientado a objetos y diagrama UML preliminar

3.1 Herramienta y criterios

Los diagramas se elaboraron con **PlantUML** (versión 1.2022.12) y se mantienen como código fuente en `docs/uml/*.puml`, de modo que cada integrante pueda revisarlos y versionarlos con Git. Se partió de las clases del semestre anterior (estereotipo «existente») y se añadieron las que exigen los nuevos módulos (estereotipo «nueva»). El modelo de dominio tiene 15 clases y 3 enumeraciones; a continuación se detallan las siete clases principales.

3.2 Diccionario de las clases de dominio principales

Conductor «existente»

Persona evaluada por el sistema. Es el centro del modelo: compone su licencia, su historial y sus renovaciones.

Elemento	Tipo / firma	Descripción
cedula	String	10 dígitos, validada con el algoritmo módulo 10.
nombreCompleto	String	Solo letras y espacios; clave del índice por nombre (M1).
fechaNacimiento	LocalDate	Debe dar una edad ≥ 18 años.
puntajeActual	double	En [0, 100]; prioridad en el ranking de riesgo (M3).
estado	EstadoConductor	ACTIVO, INACTIVO o SUSPENDIDO (enum nuevo).
getEdad()	int	Edad calculada a la fecha actual.
puedeRenovarLicencia()	boolean	Licencia vencida, conductor activo y puntaje ≥ 50.
agregarInfraccion(i)	void	Agrega el evento al historial y recalcula el puntaje.
Asociaciones	1 Licencia (composición) · 1 HistorialConduccion (composición) · 0.. *RenovacionLicencia (composición)* · *0..* Prueba · 0.. *InscripcionCurso* · *0..* Alerta	

Licencia «existente»

Permiso de conducir del titular. No existe sin su conductor, por eso se modela como composición.

Elemento	Tipo / firma	Descripción
numero	String	Formato B-001 a E-999999; clave del AVL de M1.
categoria	CategoriaLicencia	B, C o E (enum nuevo).
fechaEmision / fechaVencimiento	LocalDate	Vigencia de 5 años (DURACION_ANIOS).
estaVencida()	boolean	Compara el vencimiento con la fecha actual.
diasParaVencer()	long	Nuevo: prioridad en la cola de caducidad (M5).

Elemento	Tipo / firma	Descripción
renovarDesde(fecha)	void	Reinicia la vigencia desde la fecha dada.
Asociaciones	1 Conductor (parte de su composición)	

EventoConduccion «existente, abstracta» e Infraccion

Superclase abstracta de los hechos registrados al conductor; Infraccion es el tipo concreto que hoy registra el sistema. La herencia es natural porque toda infracción es un evento con la misma regla de deducción.

Elemento	Tipo / firma	Descripción
idEvento	String	Identificador EVT-###.
fechaHora	LocalDate	Clave del índice temporal AVL (M2); no futura ni mayor a 5 años.
ubicacion	String	Texto de la vía; se asocia con una Zona del mapa vial.
gravedad	int	De 1 a 5.
calcularPuntosDeduccion()	int	puntosBase(tipo) × gravedad.
clasificarGravedad()	String	Leve, moderado, grave o muy grave.
Asociaciones	0.. → *1 TipoEvento* · *0..* → 0..1 Zona · parte de 1 HistorialConduccion · Infraccion hereda de EventoConduccion	

HistorialConduccion «existente»

Concentra los eventos y los ajustes de puntaje de un conductor; es la fuente del recálculo que alimenta a M3.

Elemento	Tipo / firma	Descripción
eventos	List<EventoConduccion>	Eventos del conductor en orden de registro.
puntajeInicial	double	100.
puntosRecuperacion	double	Suma de los cursos aprobados.
bonificacionBuenComportamiento	double	5 puntos por año sin infracciones.
calcularPuntajeFinal()	double	clamp(0, 100, 100 − Σ deducciones + recuperación + bonificación).
Asociaciones	1 Conductor (composición) · 0..* EventoConduccion (composición)	

Curso «existente» e InscripcionCurso «nueva»

El curso define los puntos recuperables y su banco de preguntas; la inscripción es el elemento de la cola FIFO con la que M4 asigna los cupos.

Elemento	Tipo / firma	Descripción
Curso.codigo / nombre	String	CUR-101, CUR-102…; clave del AVL de cursos.
Curso.puntosRecuperacion	int	20 (básico) o 30 (avanzado).
Curso.porcentajeMinimoAprobacion	double	70 % por defecto.
Curso.cupoMaximo	int	Nuevo: cupos por convocatoria.
InscripcionCurso.fechaSolicitud	LocalDate	Orden de llegada a la cola.
InscripcionCurso.estado	String	EN_ESPERA, ASIGNADA o ATENDIDA.
asignar() / marcarAtendida()	void	Transiciones de estado de la inscripción.
Asociaciones	Curso 1 → 0.. *Pregunta (composición)* · *InscripcionCurso 0..* → 1 Conductor y 0.. → *1 Curso* · *Prueba 0..* → 1 Curso	

Zona «nueva» y TramoVial «nueva»

Modelan el mapa vial sobre el que M5 planifica los operativos: las zonas son los vértices del grafo y los tramos, sus aristas ponderadas.

Elemento	Tipo / firma	Descripción
Zona.codigo / nombre	String	Identifica el vértice (p. ej. «Av. 10 de Agosto y Colón»).
Zona.infraccionesRegistradas	int	Contador del periodo; define si es zona crítica.
esPuntoCritico(umbral)	boolean	Supera el umbral de infracciones.
TramoVial.distanciaKm	double	Peso de la arista (positivo) usado por Dijkstra.
TramoVial.bidireccional	boolean	Si el tramo puede recorrerse en ambos sentidos.
Asociaciones	TramoVial 0.. → *2 Zona* · *EventoConduccion 0..* → 0..1 Zona	

3.3 Justificación de herencia, composición y asociaciones

Se usa **composición** cuando la parte no tiene sentido sin el todo: la licencia, el historial y las renovaciones pertenecen al conductor; los eventos, al historial; las preguntas, al curso. Se usa **herencia** solo en EventoConduccion → Infraccion, porque la superclase define la regla común de deducción y deja abierta la incorporación de otros eventos (por ejemplo, siniestros) sin duplicar código. El resto son **asociaciones** simples, ya que TipoEvento, Curso o Zona existen de forma independiente y son compartidos por muchos objetos. No se forzó la agregación: ninguna relación del dominio la requiere.

3.4 Correspondencia entre módulos, clases y estructuras

Tabla 4. *Trazabilidad módulo → clases → estructura de datos*

Módulo	Clases de dominio	Gestor y estructura
M1 Conductores y licencias	Conductor, Licencia, RenovacionLicencia, CategoriaLicencia, EstadoConductor	GestorConductores · 2 árboles AVL (licencia y nombre)
M2 Eventos e infracciones	EventoConduccion, Infraccion, TipoEvento	GestorEventos · AVL por fecha + pila de deshacer
M3 Puntaje y riesgo	HistorialConduccion, NivelAlerta	GestorRiesgo · RankingRiesgoAVL (alternativa seleccionada)
M4 Recuperación de puntos	Curso, Pregunta, Prueba, EvaluacionPrueba, InscripcionCurso	GestorCursos · AVL de cursos + cola FIFO
M5 Alertas, reportes y operativos	Alerta, Licencia, Zona, TramoVial	GestorReportes · montículo binario + grafo vial

Cada módulo tiene un gestor en la capa `negocio` que encapsula su estructura de datos, y SistemaSECR los agrupa por composición. M2 y M4 notifican a M3 cada cambio de puntaje, de modo que el ranking de riesgo nunca queda desactualizado. Las figuras 1 y 2, en páginas horizontales, muestran el modelo completo; sus versiones vectoriales (SVG) están en `docs/uml/`.

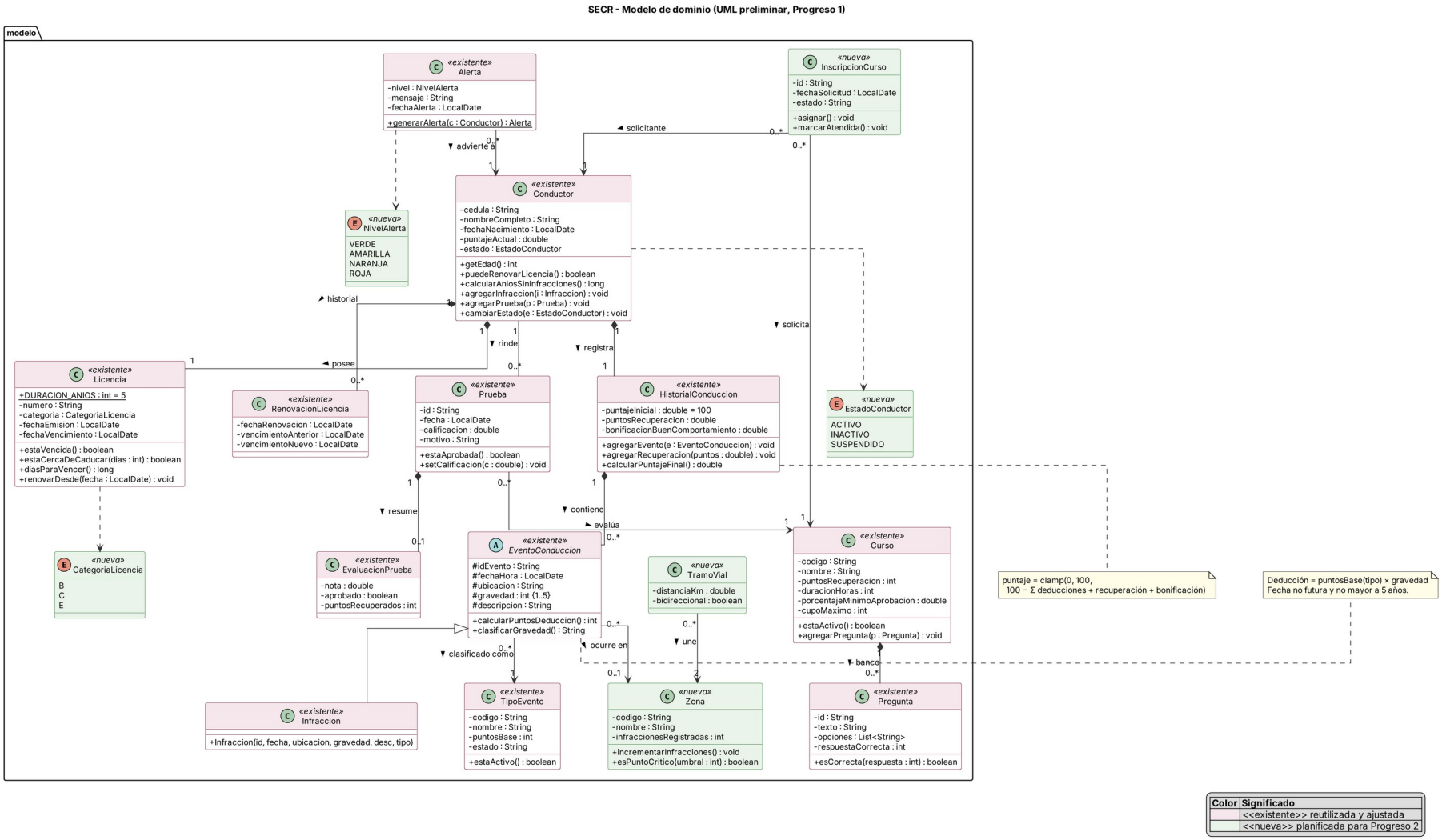


Figura 1. *Diagrama de clases preliminar: modelo de dominio de SECR (PlantUML).*

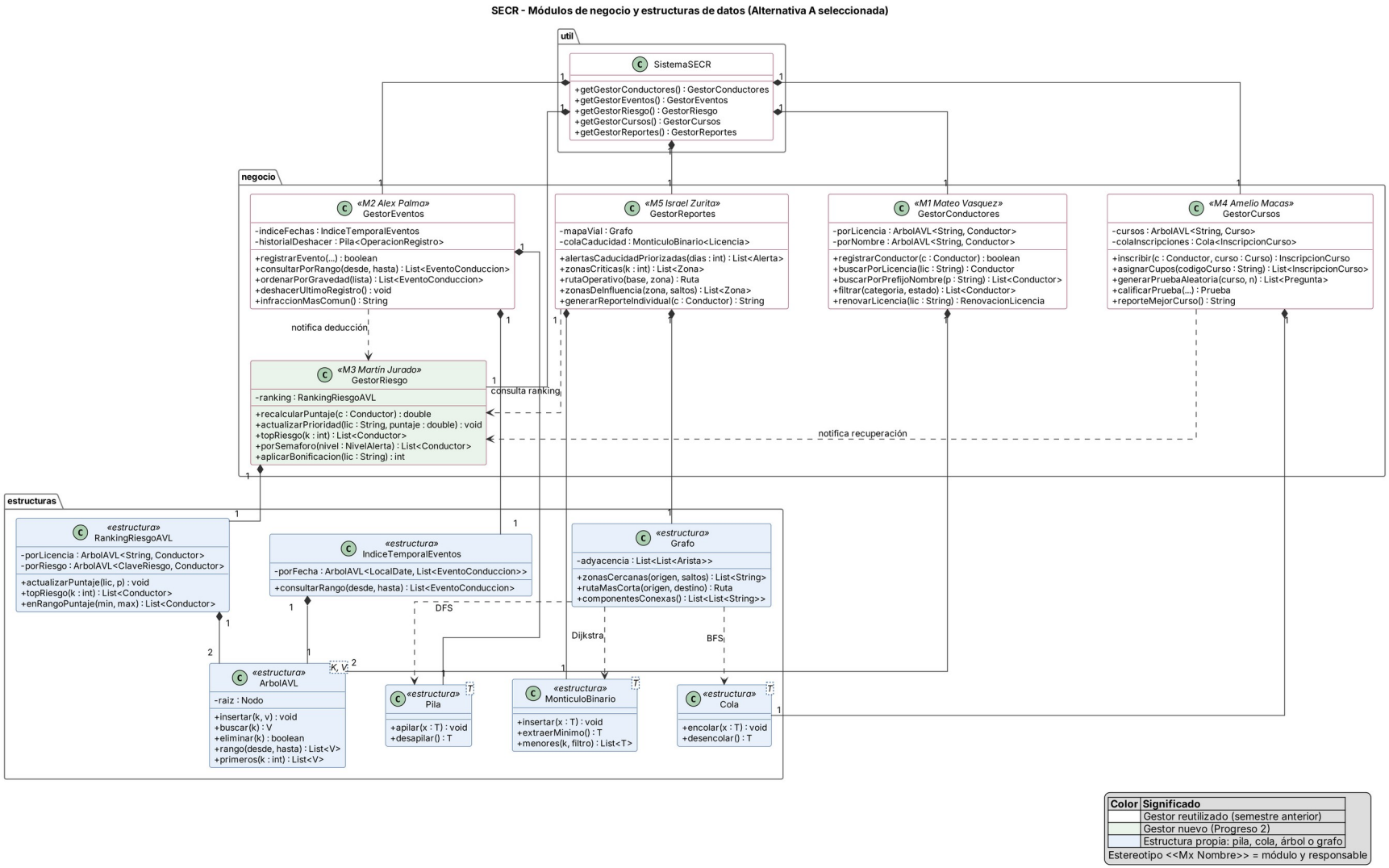


Figura 2. *Diagrama de clases: gestores por módulo y estructuras propias (pilas, colas, árboles y grafo).*

4. Alternativas basadas en estructuras de datos y algoritmos

El componente central y crítico de SECR es la **priorización de conductores por riesgo (M3)**: los puntajes cambian con cada infracción y cada curso, y el sistema debe responder en todo momento quiénes son los k conductores de mayor riesgo y quiénes están en cada nivel del semáforo. Se plantean dos alternativas viables y competitivas, construidas solo con árboles y colas. Ambas comparten un árbol AVL de licencias para localizar al conductor. Notación: n = conductores, k = resultados, m = entradas del montículo, s = entradas obsoletas encontradas.

La tabla 7 resume ambas alternativas en el formato solicitado; la tabla 8 detalla la complejidad por operación.

Tabla 5. *Alternativas planteadas (formato de la consigna)*

Alternativa	Estructura(s)	Algoritmo(s)	Operación que resuelve	Complejidad temporal	Complejidad espacial	Ventajas / limitaciones
A Árboles AVL	• Árbol AVL licencia → conductor • Árbol AVL (puntaje, licencia) → conductor, ordenado por riesgo	• Inserción y eliminación con rotaciones simples y dobles • Recorrido en orden (in-order) iterativo con pila • Consulta de rango con poda	Reubicar al conductor cuando cambia su puntaje; top-k de mayor riesgo; filtro por semáforo	• Actualizar puntaje: O(log n) (eliminar + insertar) • Top-k: O(log n + k) • Semáforo: O(log n + k), ordenado • Buscar licencia: O(log n)	O(n): exactamente 2 nodos por conductor, sin importar cuántas actualizaciones ocurran	**Ventajas:** consultas muy rápidas y siempre ordenadas; memoria constante por conductor; el mismo árbol sirve a M1, M2 y M4. **Limitaciones:** cada cambio de puntaje cuesta dos operaciones con rotaciones; implementación más delicada.
B Cola de prioridad perezosa	• Montículo binario mínimo (árbol completo en arreglo) como cola de prioridad • Árbol AVL licencia → registro con versión	• Flotar / hundir (sift-up/down) • Actualización perezosa: se inserta una entrada nueva y la anterior queda obsoleta • Compactación (heapify de Floyd) cuando obsoletas > vigentes • Top-k con frontera de índices	Las mismas tres operaciones	• Actualizar puntaje: O(log m), m ≤ 2n • Top-k: O((k + s) log(k + s)) • Semáforo: O(k + s), sin orden • Compactación: O(n) ocasional	O(n) a O(2n) entradas en el montículo + 1 nodo AVL por conductor	**Ventajas:** actualizaciones más baratas (una sola inserción en arreglo); estructura sencilla de implementar. **Limitaciones:** las entradas obsoletas encarecen las consultas y la memoria; resultados del semáforo sin orden; pausas O(n) al compactar.

Nota. Elaboración propia. Árbol AVL según Adelson-Velsky y Landis (1962) y Knuth (1998); montículo binario según Williams (1964); top-k sobre montículos según Sedgewick y Wayne (2011); complejidades según Cormen et al. (2022).

4.1 Detalle de complejidad por operación

Tabla 6. *Comparación de complejidad por operación del componente M3*

Operación	Alternativa A · AVL	Alternativa B · montículo perezoso	Frecuencia en SECR
Registrar conductor	O(log n) en 2 árboles	O(log n) AVL + O(log m) montículo	Media
Actualizar puntaje (infracción o curso)	O(log n): eliminar + insertar con rotaciones	O(log m): insertar entrada nueva	Alta
Top-k de mayor riesgo	O(log n + k)	O((k + s) log(k + s))	Muy alta (panel del analista)
Conductores en un nivel del semáforo	O(log n + k), ordenados	O(k + s), sin orden	Alta (alertas y reportes)
Buscar por licencia	O(log n)	O(log n)	Muy alta
Memoria	2n nodos, fija	n a 2n entradas + n nodos, variable	—

Ambas alternativas conviven con las demás estructuras de la solución, que no dependen de la elección: árbol AVL por fecha para el historial de eventos, pila para deshacer registros, cola FIFO para los cupos de los cursos, cola de prioridad para la caducidad de licencias y grafo ponderado del mapa vial con BFS, DFS y Dijkstra (Dijkstra, 1959).

5. Selección y fundamentación de la mejor alternativa

5.1 Evidencia experimental

Ambas alternativas se implementaron en Java detrás de la interfaz común `RankingRiesgo` (paquete `estructuras.comparacion`). Primero se verificó con pruebas automáticas (`benchmark.PruebasEstructuras`, 29 pruebas) que A y B devuelven exactamente los mismos resultados durante 4 000 actualizaciones aleatorias. Luego se midieron con `benchmark.BenchmarkAlternativas`, con datos sintéticos de semilla fija, calentamiento previo de la JVM y la mediana de cinco repeticiones (OpenJDK 21, Linux, 2 vCPU).

Tabla 7. *Resultados del benchmark con n = 100 000 conductores (mediana, en milisegundos)*

Operación	A (ms)	B (ms)	Más rápida
Registrar 100 000 conductores	288,0	204,6	B, 1,4 veces más rápida
20 000 actualizaciones de puntaje	116,0	42,2	B, 2,8 veces más rápida
500 × (actualizar puntaje + top-10)	3,05	2,06	B, 1,5 veces más rápida
1 000 consultas top-10	0,18	1,53	A, 8,5 veces más rápida
50 consultas del semáforo rojo	64,8	220,2	A, 3,4 veces más rápida
10 000 búsquedas por licencia	15,1	15,6	Empate técnico
Memoria tras n registros y 2n cambios (MB)	8,4	12,4	A usa 32 % menos

Nota. Elaboración propia a partir de `docs/benchmark/resultados.csv`, que incluye también n = 1 000 y n = 10 000. Los valores absolutos dependen del equipo; lo relevante es la proporción entre A y B.

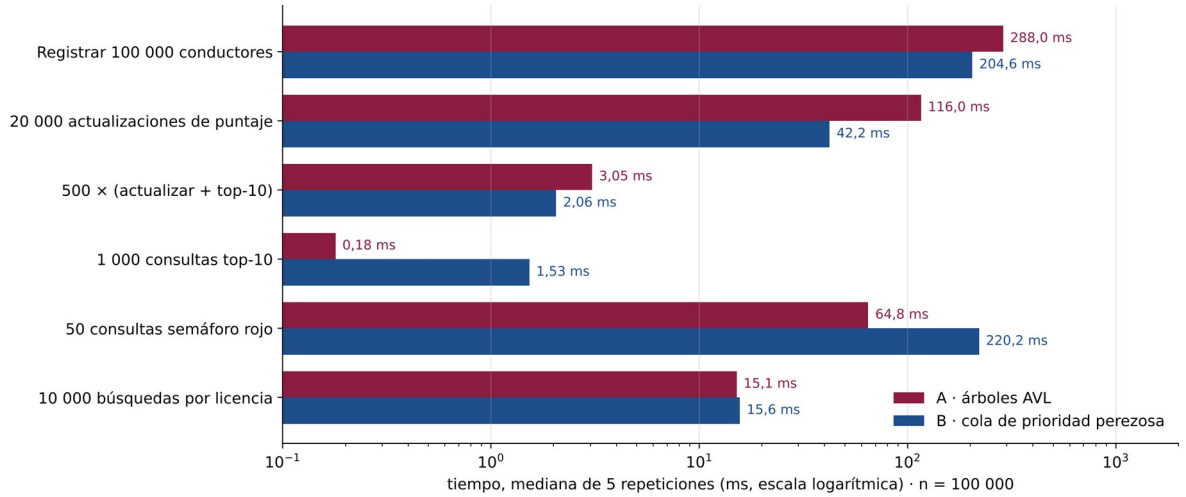


Figura 3. *Tiempo de A y B por operación con n = 100 000 (escala logarítmica).*

Lectura de los resultados. Las alternativas son realmente competitivas. B gana en las escrituras: registra 1,4 veces más rápido y actualiza puntajes 2,8 veces más rápido, porque inserta una sola entrada en un arreglo en lugar de eliminar e insertar con rotaciones. A gana en las consultas: el top-10 es 9 veces más rápido y el semáforo rojo 3,4 veces más rápido, porque B debe saltar entradas obsoletas. En la operación mixta (actualizar y consultar el top-10) B es ligeramente más rápida con 100 000 conductores y A con 1 000 y 10 000. Las búsquedas por licencia empatan porque ambas usan el mismo AVL.

Limitaciones de la medición: datos sintéticos, un solo equipo y variaciones propias de la JVM (compilación JIT y recolección de basura); por eso se reportan medianas y se interpretan proporciones, no valores exactos.

5.2 Análisis por criterios

Cada alternativa se evaluó de 1 (deficiente) a 5 (excelente). Los pesos reflejan que SECR es un sistema de consulta intensiva: el analista y los reportes de M5 consultan el ranking y el semáforo con mucha más frecuencia de la que se registran infracciones.

Tabla 8. *Matriz de decisión ponderada*

Criterio	Peso	A	B	Argumento técnico
Adecuación al problema	25 %	5	3	SECR consulta el ranking y los semáforos de forma continua y M5 necesita resultados ordenados; A los entrega ordenados por construcción. B devuelve el semáforo sin orden.
Tiempo de ejecución / complejidad	25 %	4	4	Empate: B actualiza ≈ 2,7 veces más rápido; A consulta el top-10 ≈ 8 veces y el semáforo ≈ 3 veces más rápido. Ambas son logarítmicas.
Uso de memoria RAM	10 %	5	3	A ocupa memoria fija (2 nodos por conductor); B acumula entradas obsoletas hasta compactar y usó ≈ 47 % más.
Escalabilidad	15 %	4	4	Ambas crecen de forma logarítmica de 10^3 a 10^5; B escala mejor ante ráfagas de escrituras, A ante consultas y no tiene pausas O(n).
Mantenibilidad	15 %	5	3	A reutiliza el mismo ArbolAVL genérico de M1, M2 y M4; B necesita dos estructuras sincronizadas y un control de versiones.
Dificultad razonable de implementación	10 %	3	4	Las rotaciones del AVL son más delicadas que el montículo; se mitiga con el prototipo ya probado.
Puntaje ponderado	**100 %**	**4,40**	**3,50**	**A supera a B en adecuación, memoria y mantenibilidad.**

5.3 Decisión

Se implementará la Alternativa A (dos árboles AVL), con un puntaje ponderado de 4,40 frente a 3,50. Se ajusta mejor al problema porque entrega el ranking y los semáforos ordenados con costo O(log n + k), mantiene una memoria fija por conductor sin pausas de compactación y reutiliza el mismo árbol AVL genérico que ya usan M1, M2 y M4, lo que simplifica el mantenimiento y las pruebas. Su desventaja, actualizaciones unas 2,7 veces más lentas que B, es aceptable: 20 000 actualizaciones con 100 000 conductores toman alrededor de un décimo de segundo.

Condición de revisión: si en el futuro SECR recibiera infracciones de forma masiva y automática (por ejemplo, desde cámaras de fotomulta), con muchas más escrituras que consultas, la Alternativa B pasaría a ser preferible y podría incorporarse detrás de la misma interfaz RankingRiesgo sin cambiar los demás módulos.

6. Plan preliminar de implementación y gestión

6.1 Módulos y componentes asignados

Tabla 9. *Componentes a implementar en el Progreso 2 por integrante*

Integrante	Módulo	Componentes a implementar
Mateo Vasquez	M1	Integrar los AVL de licencia y de nombre en GestorConductores; búsqueda por prefijo con consulta de rango; filtros por recorrido en orden; enums EstadoConductor y CategoriaLicencia.
Alex Palma	M2	Integrar IndiceTemporalEventos (AVL por fecha); consulta por rango en PanelEventos; pila de deshacer; ordenamiento por gravedad con OrdenamientoMezcla.
Martín Jurado	M3	Crear GestorRiesgo con RankingRiesgoAVL; notificaciones desde M2 y M4; semáforo con NivelAlerta; panel de ranking top-k.
Amelio Macas	M4	Clase InscripcionCurso y cola FIFO con cupos en GestorCursos; prueba aleatoria con Fisher-Yates; flujo de calificación; administración del repositorio.
Israel Zurita	M5	Clases Zona y TramoVial; mapa vial con Grafo (BFS, DFS y Dijkstra); alertas de caducidad con MonticuloBinario; reportes; mantenimiento del benchmark y las pruebas.

6.2 Estructura del repositorio Git

El repositorio (https://github.com/isam1725/SECR-Progreso1) ya usa la estructura estándar `src/main/java`, contiene el código reutilizado sin tablas hash, las estructuras propias, el benchmark, los diagramas y el README con las instrucciones de compilación y ejecución:

```
SECR-Progreso1/
├── README.md                    Descripción, integrantes, compilación y ejecución
├── .gitignore                   Excluye out/, *.class y archivos del IDE
├── docs/
│   ├── informe/                 Informe técnico del Progreso 1 (PDF)
│   ├── uml/                     Diagramas .puml y exportados .png/.svg
│   └── benchmark/               resultados.csv, grafico_benchmark.png
└── src/main/java/
    ├── modelo/                  Clases de dominio (reutilizadas)
    ├── negocio/                 Gestores de cada módulo (ahora sobre ArbolAVL)
    ├── interfaz/                Interfaz gráfica Swing (reutilizada)
    ├── util/                    SistemaSECR (fachada)
    ├── estructuras/             ArbolAVL, Pila, Cola, MonticuloBinario, Grafo,
    │   │                        IndiceTemporalEventos, OrdenamientoMezcla
    │   └── comparacion/         RankingRiesgo: alternativas A (AVL) y B (montículo)
    └── benchmark/               PruebasEstructuras, BenchmarkAlternativas
```

Flujo de trabajo: rama `main` (entregas), rama `develop` (integración) y una rama `feature/` por módulo; todo cambio entra mediante *pull request* revisado por otro integrante y con las pruebas automáticas en verde.

6.3 Cronograma hacia el Progreso 2

Tabla 10. *Cronograma preliminar*

Semana	Actividad	Responsables
1	Incorporar las observaciones del docente sobre el Progreso 1; crear ramas; migrar estados a enums.	Todos
2	Índices AVL de M1 y M2; pila de deshacer.	M. Vasquez, A. Palma
3	GestorRiesgo con RankingRiesgoAVL y notificaciones de M2 y M4.	M. Jurado
4	Cola de inscripción con cupos (M4); grafo vial, Dijkstra y alertas de caducidad (M5).	A. Macas, I. Zurita
5	Integración en la interfaz Swing, pruebas de regresión y nueva ejecución del benchmark.	Todos
6	Ajustes finales, actualización de UML e informe, ensayo de la presentación.	Todos

6.4 Riesgos técnicos y acciones de mitigación

Tabla 11. *Riesgos técnicos identificados*

Riesgo	Prob.	Impacto	Acción de mitigación
Rotaciones del AVL mal implementadas que rompen el orden o el balance sin que se note en pruebas pequeñas.	Media	Alto	Método esAVLValido() que verifica orden y alturas; prueba con 20 000 operaciones aleatorias contra una referencia y con 100 000 claves crecientes (altura obtenida: 17).
Desincronización de los dos árboles de M3 al cambiar un puntaje (la clave vieja no se elimina).	Media	Alto	Toda modificación pasa por RankingRiesgo.actualizarPuntaje, que elimina la clave anterior antes de cambiar el puntaje; prueba de equivalencia A = B en 4 000 actualizaciones.
Desbordamiento de pila (StackOverflowError) en recorridos recursivos de árboles o grafos grandes.	Baja	Alto	La altura del AVL está acotada por $1{,}44 \cdot \log_2 n$; los recorridos en orden, el BFS y el DFS son iterativos con Cola y Pila propias.
Datos del mapa vial inválidos: distancias negativas o zonas desconectadas que hacen fallar Dijkstra.	Media	Medio	Validar distancias positivas al crear tramos; detectar zonas aisladas con componentesConexas() y reportar 'sin ruta' en vez de fallar.
La sustitución de las tablas hash heredadas rompe pantallas de la interfaz Swing.	Media	Medio	Mismos métodos públicos en los gestores; pruebas automáticas de los gestores y prueba manual de cada panel antes de fusionar.
Conflictos de fusión en Git al trabajar cinco personas sobre los gestores.	Alta	Medio	Una rama por módulo, commits pequeños, pull requests revisados y fusión frecuente a develop.

6.5 Declaración de uso de herramientas de inteligencia artificial

El equipo utilizó una herramienta de IA generativa como apoyo, conforme a lo permitido por la consigna. Todo contenido incorporado fue revisado, adaptado y comprendido por los integrantes, quienes asumen la responsabilidad académica y técnica sobre el código, los diagramas, las referencias y las decisiones de diseño.

Tabla 12. *Uso de herramientas de IA*

Herramienta	Uso realizado	Resultado incorporado	Forma de verificación
Claude (Anthropic)	Apoyo para estructurar el informe a partir de la consigna y del código del semestre anterior; revisión de redacción.	Borradores de las secciones 1 a 6, ajustados por el equipo.	Lectura y corrección por cada responsable de módulo; contraste de cifras y referencias con las fuentes originales (enlaces y DOI).
Claude (Anthropic)	Apoyo en el código Java de ArbolAVL, Pila, Cola, MonticuloBinario y Grafo, de las alternativas A y B y del benchmark; sustitución de las tablas hash heredadas.	Paquetes estructuras, estructuras.comparacion y benchmark; gestores modificados.	Compilación y ejecución de 29 pruebas automáticas; revisión del código y de las complejidades por el equipo; repetición del benchmark.
Claude (Anthropic)	Borrador de los diagramas UML en PlantUML y de la presentación.	Diagramas de las figuras 1 y 2; diapositivas.	Revisión de clases, multiplicidades y correspondencia con los módulos; ensayo de la exposición.

Referencias

Adelson-Velsky, G. M., & Landis, E. M. (1962). An algorithm for the organization of information. *Soviet Mathematics Doklady, 3*, 1259–1263.

Aguas Bucheli, L. F. (2024). *Estructuras de datos para todos: Ejercicios y teoría con Java*. Our Knowledge Publishing.

Asamblea Nacional del Ecuador. (2008, 7 de agosto). *Ley Orgánica de Transporte Terrestre, Tránsito y Seguridad Vial* (Registro Oficial Suplemento 398, con reformas).

Castillo-Manzano, J. I., & Castro-Nuño, M. (2012). Driving licenses based on points systems: Efficient road safety strategy or latest fashion in global transport policy? A worldwide meta-analysis. *Transport Policy, 21*, 191–201. https://doi.org/10.1016/j.tranpol.2012.02.003

Castillo-Manzano, J. I., Castro-Nuño, M., & Pedregal, D. J. (2010). An econometric analysis of the effects of the penalty points system driver's license in Spain. *Accident Analysis & Prevention, 42*(4), 1310–1319. https://doi.org/10.1016/j.aap.2010.02.009

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*, 269–271. https://doi.org/10.1007/BF01386390

Durstenfeld, R. (1964). Algorithm 235: Random permutation. *Communications of the ACM, 7*(7), 420. https://doi.org/10.1145/364520.364540

El Universo. (2025, 22 de enero). *Durante 2024 aumentaron los siniestros de tránsito en Ecuador, pero se redujo la cantidad de fallecidos y lesionados, según la ANT*. https://www.eluniverso.com/noticias/ecuador/siniestros-de-transito-en-ecuador-durante-el-2024-nota/

Knuth, D. E. (1998). *The art of computer programming: Vol. 3. Sorting and searching* (2nd ed.). Addison-Wesley.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Williams, J. W. J. (1964). Algorithm 232: Heapsort. *Communications of the ACM, 7*(6), 347–348.

World Health Organization. (2023). *Global status report on road safety 2023*. https://www.who.int/publications/i/item/9789240086517

Anexo A. Reproducción de los resultados

Desde la raíz del repositorio, con JDK 17 o superior:

```
javac -encoding UTF-8 -d out $(find src/main/java -name "*.java")
java -cp out benchmark.PruebasEstructuras           # 29 pruebas correctas
java -Xmx3g -cp out benchmark.BenchmarkAlternativas  # genera resultados.csv
java -cp out interfaz.Interfaz                       # aplicación gráfica
```